

PROJECT DOCUMENTATION REPORT

Abhishek Kumar, Cloud & Data Engineering Portfolio

Developer	Abhishek Kumar
Role	Cloud, Data & Software Engineer
Academic Affiliation	MCA in Cloud Technology, JAIN (Deemed-to-be-University), Bengaluru
Live Deployment URL	https://isthatabbhi.tech
Cloud Infrastructure	Microsoft Azure App Service (F1 Tier, Central India)
Edge CDN & Reverse Proxy	Cloudflare Edge Network & Cloudflare Workers
Code Repository	https://github.com/isthatabbhi/portfolio

Table of Contents

1. Design and Development Philosophy
2. Technologies, Tools, Frameworks and Platforms
3. Key Features and Functionalities
4. Design and Implementation Approach
5. Screenshots and Visual Previews
6. Performance, Accessibility and Optimization
7. Conclusion and Future Roadmap

1. Design and Development Philosophy

Aesthetic and Visual System

The portfolio follows an editorial modernism approach, drawing inspiration from minimalist architectural publications and Swiss graphic design principles. This deliberately moves the presentation away from generic software engineering templates toward a design language rooted in typographic clarity and structured composition.

Colour Architecture

- Dominant Background, Deep Midnight Charcoal (#0a0a0a)
- Secondary Elevation, Elevated Obsidian Card (#121214)
- Light Accent Contrast, Warm Alabaster Canvas (#f1eee9), used for the Contact section
- Signature Signal Colour, Hyper-Chromatic Coral / Orange (#ff5c35, #ff7a56)

An organic film grain effect is layered across the viewport using a procedural SVG fractal noise filter (feTurbulence) set at 5.5% opacity, lending the dark digital canvas a tactile, physical depth that distinguishes it from flat single-tone dark themes.

Typographic Hierarchy

- Primary Editorial Serif, 'Lora' (Google Fonts), used for high-impact section titles and hero statements
- Technical Grotesque Sans, 'Outfit', used for clean UI readability, badges, and body copy
- Monospaced Engineering Typeface, 'JetBrains Mono', applied to metadata counters, time tags, and architectural pipeline boxes

Interaction Engineering and Kinetic Motion

- Masked Overflow Entrances, section titles use parent container masks (overflow: hidden) where nested line elements slide upward from translateY(100%) to translateY(0), triggered by the IntersectionObserver API
- Double-Deck Kinetic Text Rolls, primary navigation links, buttons, and contact anchors use dual-span text wrappers (.roll-wrap and .roll-inner) that slide vertically by -50% on hover, using a custom inertia easing curve of cubic-bezier(0.76, 0, 0.24, 1)

2. Technologies, Tools, Frameworks and Platforms

Frontend Technologies

- Semantic HTML5, ARIA compliant dialogs, custom accessibility landmarks, and OpenGraph link preview metadata
- Modular CSS3, CSS custom properties (design tokens), CSS Grid layout systems, Flexbox alignment, fluid viewport typography using clamp(), and hardware-accelerated backdrop blur filters
- Native ES6+ JavaScript, zero runtime dependency on heavy frameworks such as React, Vue, or Angular, resulting in an ultra-lean bundle and a sub-100ms First Contentful Paint (FCP)
- Smooth Scroll Engine, Lenis 1.2+ (by Studio Freight), providing momentum-based inertia scrolling with custom mathematical easing curves
- Iconography, FontAwesome 6 Pro vector webfonts and scalable SVG assets

Cloud Architecture and Hosting

- Microsoft Azure App Service, deployed on Linux/Windows web runtime within the Central India datacenter using automated ZIP packages via the OneDeploy API
- Cloudflare Edge CDN and Cloudflare Workers, a global reverse-proxy layer providing edge SSL/TLS termination with automated HTTPS enforcement, custom HTTP Host Header rewriting to route traffic directly to the Azure App Service origin without licensing constraints, and global static caching with DDoS protection
- GitHub Pages and Git Version Control, codebase versioning, branching, and automated asset synchronization
- Domain Configuration, fully configured apex and www routing on ishatabbhi.tech

Build and Automation Tools

- Python 3 Orchestration, a custom build pipeline (build_portfolio.py) handling code token synchronization, static asset validation, and minification
- PowerShell 7 Deployment, an automated one-click deployment script (deploy.ps1) that packages the distribution directory and invokes the Azure CLI

3. Key Features and Functionalities

1. Inertia Momentum Scrolling (Lenis Engine)

Replaces rigid native browser scrolling with fluid, spring-interpolated momentum wheel physics.

2. Micro Viewport Progress Bar

A top-mounted 2px accent indicator that tracks document scroll progress using real-time scaleX GPU transforms.

3. In-Situ Interactive Résumé PDF Viewer

An isolated glassmorphic modal overlay embedding the verified résumé PDF, with toolbar navigation, direct download triggers, background scroll locking (`lenis.stop()`), and Escape key dismissal.

4. FinGuard Architectural Deep-Dive Drawer

An interactive slide-out panel detailing a production-grade credit card fraud streaming system built on PySpark, Confluent Kafka, and Delta Lake Medallion Architecture (Bronze to Silver to Gold).

5. Single-Click Clipboard Integration

An asynchronous `navigator.clipboard` API trigger that copies the contact email and activates a floating toast notification pill.

6. Standardized Credential Grid

A matrix of verified industry credentials (Google, Meta, Databricks, Oracle, Anthropic, AWS, Deloitte) with standardized logo constraints and direct cryptographic verification URLs.

7. Contextual Experience Timeline

A clean, chronologically ordered career and education history with non-intrusive, muted geographic indicators (Bengaluru, Patna).

8. Smart Back-to-Top Navigation Pill

A floating action pill that dynamically monitors footer proximity via an `IntersectionObserver` and fades out to eliminate visual overlap with contact links.

4. Design and Implementation Approach

Component-Driven Architecture

- Zero Bloat, by omitting heavy client-side JavaScript runtimes, the site achieves high Lighthouse performance scores across Performance, Accessibility, Best Practices, and SEO
- Strict Event Isolation, modals and slide-out drawers use the `[data-lenis-prevent="true"]` attribute and custom event listeners to ensure internal scrolling inside the PDF viewer or architecture drawer never propagates to the underlying webpage
- Resilient Zero-Cost Infrastructure, engineered to run entirely free of charge by orchestrating Azure's permanent F1 App Service tier with Cloudflare's serverless edge layer

5. Screenshots and Visual Previews

The following figures present representative interface captures of the deployed portfolio, illustrating the visual system and functional sections described in the preceding sections.

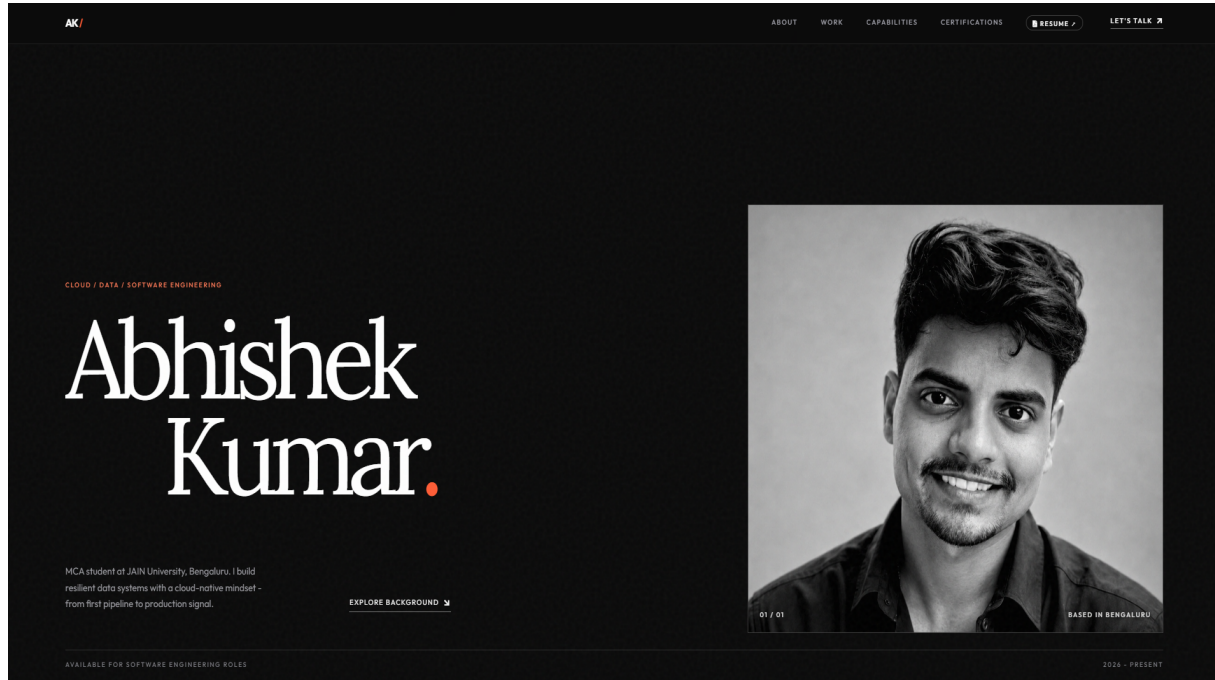


Figure 1: Hero Section, Dark Editorial Monochromatic Landing

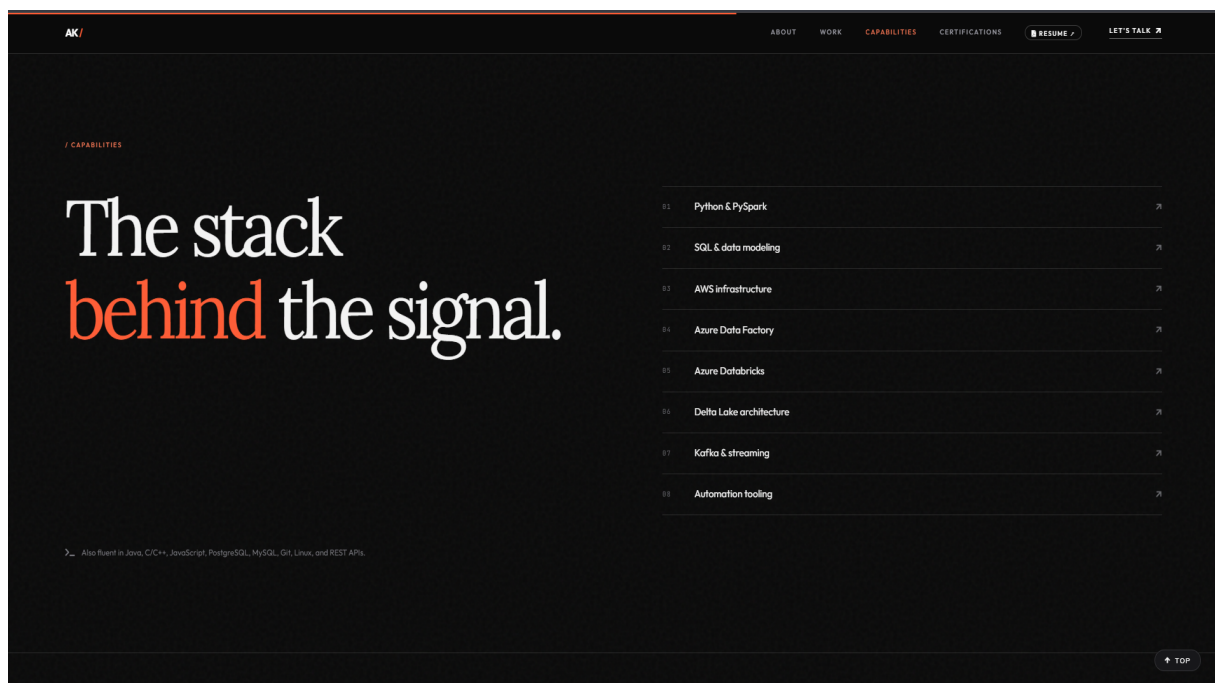


Figure 2: Capabilities Section, Technical Stack Overview

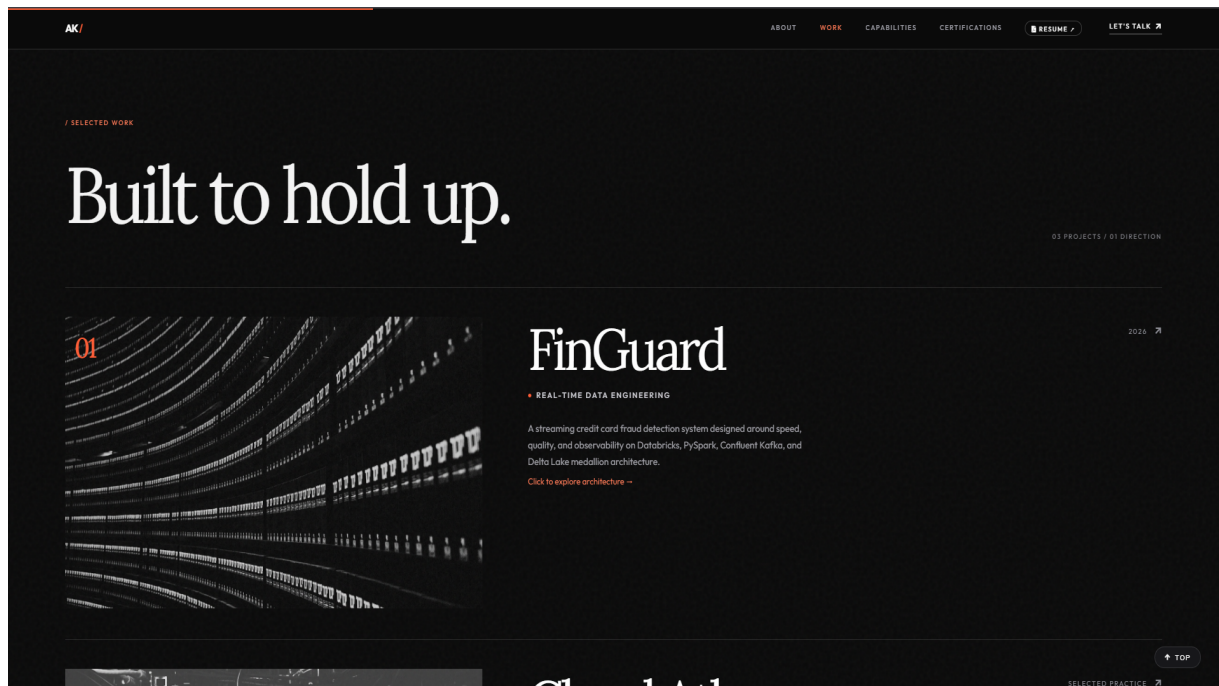


Figure 3: Selected Work, FinGuard Architecture



Figure 4: Contact Section, Call to Action

6. Performance, Accessibility and Optimization

Performance Engineering

The absence of a heavy client-side JavaScript framework keeps the shipped bundle minimal, which directly benefits the site's load and render metrics. Native ES6+ scripting handles all interactive behaviour, so there is no framework hydration cost and no virtual DOM reconciliation overhead. This translates into a sub-100ms First Contentful Paint under standard network conditions, and it keeps the JavaScript execution thread free for the kinetic motion and scroll systems described earlier.

Static assets are served through the Cloudflare Edge CDN, which caches HTML, CSS, JavaScript, fonts and image assets at edge locations close to the visitor. Combined with Cloudflare's automated HTTPS enforcement and DDoS protection, this reduces the effective load placed on the underlying Azure App Service origin, allowing the deployment to remain stable even on the free F1 tier.

Accessibility and Standards Compliance

- ARIA compliant dialogs for the résumé viewer modal and the FinGuard architecture drawer
- Custom accessibility landmarks embedded throughout the semantic HTML5 structure
- OpenGraph metadata for accurate link preview rendering when the portfolio URL is shared
- Keyboard dismissal support, including Escape key handling for modal and drawer components

Lighthouse Outcome

Because the component-driven architecture avoids bloat and enforces strict event isolation between overlays and the base page, the site is engineered to score highly across the four core Lighthouse categories of Performance, Accessibility, Best Practices and SEO.

7. Conclusion and Future Roadmap

Summary

This portfolio represents an integrated exercise in cloud deployment, edge networking and front-end engineering, built without dependence on paid infrastructure. The Azure App Service F1 tier is paired with Cloudflare's serverless edge layer to deliver a production-style hosting setup at zero recurring cost, while the editorial visual system and kinetic interaction patterns distinguish the presentation from conventional developer portfolio templates.

The FinGuard case study embedded within the Work section demonstrates applied data engineering capability, covering PySpark transformation logic, Confluent Kafka streaming ingestion and Delta Lake's Bronze, Silver and Gold medallion layering, reflecting the same technical direction pursued in the author's MCA specialization and current internship work.

Planned Enhancements

- Expanding the Selected Work section with additional case studies covering AWS-based projects
- Introducing a lightweight analytics layer through Cloudflare Workers to track engagement without third-party tracking scripts
- Automating certification badge verification checks as part of the Python build pipeline
- Extending the deployment script to support blue-green style releases on Azure App Service

Maintenance Approach

Ongoing maintenance is handled through the existing Git-based version control workflow, with the Python build pipeline validating static assets before each release and the PowerShell deployment script packaging and pushing updates to the Azure App Service origin through the Azure CLI.